

Programación concurrente y paralela

Introducción

Condiciones generales

- **Docentes:**

- Julián Zylber (Ay 1) - jzylber [[AT]] dc.uba.ar
- Jorge Szabo (Ay 1) - jorgecszabo [[AT]] gmail.com
- Tomás Chimenti (JTP) - tach.365 [[AT]] gmail.com
- Pablo Terlisky (JTP) - terlisky [[AT]] dc.uba.ar
- Hernán Melgratti (Profesor) - hmelgra [[AT]] dc.uba.ar

- **Horario:**

- Teóricas: Lunes de 17 a 22
- Prácticas: Miércoles de 17 a 22

- Bibliografía:
 - Textos: no hay un texto principal. Referencias en la página web
 - Publicaciones relacionadas
 - Diapositivas de clases
- Página web en capus: Información al día del curso,
- Comunicación: En el campus

Programación: Enfoque clásico (secuencial)

Programas = Algoritmos + Estructuras de datos (N. Wirth)

- **Datos:** Representación de la información en la memoria de la computadora.
- **Algoritmos:** Secuencia de instrucciones que describe cómo transformar datos (entradas en salidas).

Programación: Enfoque clásico (secuencial)

Programas = Algoritmos + Estructuras de datos (N. Wirth)

- **Datos:** Representación de la información en la memoria de la computadora.
- **Algoritmos:** Secuencia de instrucciones que describe cómo transformar datos (entradas en salidas).

Desmenuzando

- **Programa:** Transforma datos de entrada en datos de salida.

Programación: Enfoque clásico (secuencial)

Programas = Algoritmos + Estructuras de datos (N. Wirth)

- **Datos:** Representación de la información en la memoria de la computadora.
- **Algoritmos:** Secuencia de instrucciones que describe cómo transformar datos (entradas en salidas).

Desmenuzando

- **Programa:** Transforma datos de entrada en datos de salida.
- **Secuencialidad:** El orden de ejecución de los pasos es **total**.
 - La ejecución de las instrucciones no se solapa.

Semántica denotacional de un programa secuencial

- Un programa describe una **transformación de estados**.

Semántica denotacional de un programa secuencial

- Un programa describe una **transformación de estados**.
- El **Estado** asigna valores (Val) a variables (Var) como abstracción de la memoria:

Semántica denotacional de un programa secuencial

- Un programa describe una **transformación de estados**.
- El **Estado** asigna valores (Val) a variables (Var) como abstracción de la memoria:
 - Conjunto de estados: $\Sigma = \text{Var} \rightarrow \text{Val}$

Semántica denotacional de un programa secuencial

- Un programa describe una **transformación de estados**.
- El **Estado** asigna valores (Val) a variables (Var) como abstracción de la memoria:
 - Conjunto de estados: $\Sigma = \text{Var} \rightarrow \text{Val}$
 - Un estado específico: $\sigma, s \in \Sigma$

Semántica denotacional de un programa secuencial

- Un programa describe una **transformación de estados**.
- El **Estado** asigna valores (Val) a variables (Var) como abstracción de la memoria:
 - Conjunto de estados: $\Sigma = \text{Var} \rightarrow \text{Val}$
 - Un estado específico: $\sigma, s \in \Sigma$
- El significado de un programa P , denotado como $\llbracket P \rrbracket$, es una **función parcial** de estado en estado:

$$\llbracket P \rrbracket : \Sigma \hookrightarrow \Sigma$$

IMP (Pequeño lenguaje imperativo)

Sintaxis

$x \in \text{Var}$	(Variables)
$n \in \text{Int}$	(Enteros)
$A ::= n \mid x \mid A + A \mid A - A \mid A \times A$	(Expr. Arit.)
$B ::= \text{true} \mid \text{false} \mid A = A \mid A \leq A \mid \neg B \mid B \wedge B$	(Expr. Bool.)
$C ::= \text{skip} \mid x := A \mid C; C \mid \text{if } B \text{ then } C \text{ else } C \mid \text{while } B \text{ do } C$	(Comandos)

Expresiones Aritméticas: $\llbracket A \rrbracket : \Sigma \rightarrow \text{Num}$

$$\llbracket n \rrbracket = \lambda\sigma.n$$

$$\llbracket x \rrbracket = \lambda\sigma.\sigma \ x$$

$$\llbracket A_1 + A_2 \rrbracket = \lambda\sigma.\llbracket A_1 \rrbracket\sigma + \llbracket A_2 \rrbracket\sigma$$

$$\llbracket A_1 - A_2 \rrbracket = \lambda\sigma.\llbracket A_1 \rrbracket\sigma - \llbracket A_2 \rrbracket\sigma$$

$$\llbracket A_1 \times A_2 \rrbracket = \lambda\sigma.\llbracket A_1 \rrbracket\sigma \times \llbracket A_2 \rrbracket\sigma$$

¹Asumimos cálculo lambda con enteros, booleanos y fix

Expresiones Aritméticas: $\llbracket A \rrbracket : \Sigma \rightarrow \text{Num}$

$$\llbracket n \rrbracket = \lambda\sigma.n$$

$$\llbracket x \rrbracket = \lambda\sigma.\sigma \ x$$

$$\llbracket A_1 + A_2 \rrbracket = \lambda\sigma.\llbracket A_1 \rrbracket\sigma + \llbracket A_2 \rrbracket\sigma$$

$$\llbracket A_1 - A_2 \rrbracket = \lambda\sigma.\llbracket A_1 \rrbracket\sigma - \llbracket A_2 \rrbracket\sigma$$

$$\llbracket A_1 \times A_2 \rrbracket = \lambda\sigma.\llbracket A_1 \rrbracket\sigma \times \llbracket A_2 \rrbracket\sigma$$

Expresiones Booleanas: $\llbracket B \rrbracket : \Sigma \rightarrow \{\text{true}, \text{false}\}$

$$\llbracket \text{true} \rrbracket = \lambda\sigma.\text{true}$$

$$\llbracket \text{false} \rrbracket = \lambda\sigma.\text{false}$$

$$\llbracket A_1 = A_2 \rrbracket = \lambda\sigma.(\llbracket A_1 \rrbracket\sigma = \llbracket A_2 \rrbracket\sigma)$$

$$\llbracket A_1 \leq A_2 \rrbracket = \lambda\sigma.(\llbracket A_1 \rrbracket\sigma \leq \llbracket A_2 \rrbracket\sigma)$$

$$\llbracket \neg B \rrbracket = \lambda\sigma.\neg(\llbracket B \rrbracket\sigma)$$

$$\llbracket B_1 \wedge B_2 \rrbracket = \lambda\sigma.(\llbracket B_1 \rrbracket\sigma \wedge \llbracket B_2 \rrbracket\sigma)$$

¹Asumimos cálculo lambda con enteros, booleanos y fix

Comandos: $\llbracket C \rrbracket : \Sigma \hookrightarrow \Sigma$

$$\llbracket \text{skip} \rrbracket = \lambda\sigma.\sigma$$

$$\llbracket C_1; C_2 \rrbracket = \llbracket C_2 \rrbracket \circ \llbracket C_1 \rrbracket$$

$$\llbracket x := A \rrbracket = \lambda\sigma.\sigma[x \mapsto \llbracket A \rrbracket\sigma]$$

$$\llbracket \text{if } B \text{ then } C_1 \text{ else } C_2 \rrbracket = \lambda\sigma.\text{if } \llbracket B \rrbracket\sigma \text{ then } \llbracket C_1 \rrbracket\sigma \text{ else } \llbracket C_2 \rrbracket\sigma$$

$$\llbracket \text{while } B \text{ do } C \rrbracket = \text{fix } (\lambda f.\lambda\sigma.\text{if } \llbracket B \rrbracket\sigma \text{ then } f(\llbracket C \rrbracket\sigma) \text{ else } \sigma)$$

²Asumimos cálculo lambda con enteros, booleanos y fix

Terminación y Correctitud

- La denotación para expresiones son funciones totales.

Terminación y Correctitud

- La denotación para expresiones son funciones totales.
 - Su evaluación siempre termina.

Terminación y Correctitud

- La denotación para expresiones son funciones totales.
 - Su evaluación siempre termina.
- La denotación para comandos es una función parcial.

$C \stackrel{\text{def}}{=} \text{while true do skip}$

$$\llbracket C \rrbracket = \text{fix}(\lambda f. \lambda \sigma. \text{if } \llbracket \text{true} \rrbracket \sigma \text{ then } f(\llbracket \text{skip} \rrbracket \sigma) \text{ else } \sigma) = \text{fix}(\lambda f. \lambda \sigma. f \sigma)$$

El punto fijo está indefinido ($\llbracket C \rrbracket = \perp$), lo que muestra que la ejecución no termina para ningún estado (entrada).

Terminación y Correctitud

- La denotación para expresiones son funciones totales.
 - Su evaluación siempre termina.
- La denotación para comandos es una función parcial.

$C \stackrel{\text{def}}{=} \text{while true do skip}$

$$\llbracket C \rrbracket = \text{fix}(\lambda f. \lambda \sigma. \text{if } \llbracket \text{true} \rrbracket \sigma \text{ then } f(\llbracket \text{skip} \rrbracket \sigma) \text{ else } \sigma) = \text{fix}(\lambda f. \lambda \sigma. f \sigma)$$

El punto fijo está indefinido ($\llbracket C \rrbracket = \perp$), lo que muestra que la ejecución no termina para ningún estado (entrada).

Correctitud

Un programa es **correcto** solo si su semántica es una función **total** sobre las entradas válidas (garantiza la terminación).

Determinismo

- El significado de un programa se modela a través de una **función**.

Determinismo

- El significado de un programa se modela a través de una **función**.
- En consecuencia la ejecución de programas es **determinística**:

Determinismo

- El significado de un programa se modela a través de una **función**.
- En consecuencia la ejecución de programas es **determinística**:
 - Si la ejecución termina para un estado σ , el estado resultante σ' es **único**.

Determinismo

- El significado de un programa se modela a través de una **función**.
- En consecuencia la ejecución de programas es **determinística**:
 - Si la ejecución termina para un estado σ , el estado resultante σ' es **único**.
 - Formalmente: $\forall \sigma, \sigma_1, \sigma_2 \in \Sigma, \llbracket C \rrbracket \sigma = \sigma_1 \wedge \llbracket C \rrbracket \sigma = \sigma_2 \implies \sigma_1 = \sigma_2$.

Enfoque clásico: Equivalencia

Equivalencia

Equivalencia

- Dos programas son equivalentes si denotan a la misma función.
- C_1 y C_2 son equivalentes ($C_1 \simeq C_2$) si $\llbracket C_1 \rrbracket = \llbracket C_2 \rrbracket$.

Enfoque clásico: Equivalencia

Equivalencia

- Dos programas son equivalentes si denotan a la misma función.
- C_1 y C_2 son equivalentes ($C_1 \simeq C_2$) si $\llbracket C_1 \rrbracket = \llbracket C_2 \rrbracket$.

Ejemplo

- Considerar $C_1 \stackrel{\text{def}}{=} x := 1$ y $C_2 \stackrel{\text{def}}{=} x := 0; x := x + 1$.
- Luego $\llbracket C_1 \rrbracket = \lambda\sigma.\sigma[x \mapsto 1]$
 $\llbracket C_2 \rrbracket = \lambda\sigma.\sigma[x \mapsto \sigma x + 1] \circ \lambda\sigma.\sigma[x \mapsto 0] = \lambda\sigma.\sigma[x \mapsto 1]$

Enfoque clásico: Equivalencia

Equivalencia

- Dos programas son equivalentes si denotan a la misma función.
- C_1 y C_2 son equivalentes ($C_1 \simeq C_2$) si $\llbracket C_1 \rrbracket = \llbracket C_2 \rrbracket$.

Ejemplo

- Considerar $C_1 \stackrel{\text{def}}{=} x := 1$ y $C_2 \stackrel{\text{def}}{=} x := 0; x := x + 1$.
- Luego $\llbracket C_1 \rrbracket = \lambda\sigma.\sigma[x \mapsto 1]$
 $\llbracket C_2 \rrbracket = \lambda\sigma.\sigma[x \mapsto \sigma x + 1] \circ \lambda\sigma.\sigma[x \mapsto 0] = \lambda\sigma.\sigma[x \mapsto 1]$
- C_1 y C_2 son equivalentes porque $\llbracket C_1 \rrbracket = \llbracket C_2 \rrbracket$

Enfoque clásico: La equivalencia es una congruencia

Principio de substitutividad

Si dos programas son equivalentes, se puede reemplazar uno por otro dentro de un programa más grande sin alterar su significado.

$P_1 = x := 1$ y $P_2 = x := 0; x := x + 1$

- Como P_1 y P_2 son equivalentes ($P_1 \simeq P_2$), para todo P valen
-

Enfoque clásico: La equivalencia es una congruencia

Principio de sustitutividad

Si dos programas son equivalentes, se puede reemplazar uno por otro dentro de un programa más grande sin alterar su significado.

$P_1 = x := 1$ y $P_2 = x := 0; x := x + 1$

- Como P_1 y P_2 son equivalentes ($P_1 \simeq P_2$), para todo P valen
 - $P; P_1 \simeq P; P_2$
 - $P_1; P \simeq P_2; P$

Sintaxis

$\mathcal{C} ::= [\cdot]$	(Agujero / Hueco)
$\mathcal{C}; \mathcal{C} \mid \mathcal{C}; \mathcal{C}$	(Secuencia)
$\text{if } B \text{ then } \mathcal{C} \text{ else } \mathcal{C}$	(Bifurcación Izquierda)
$\text{if } B \text{ then } \mathcal{C} \text{ else } \mathcal{C}$	(Bifurcación Derecha)
$\text{while } B \text{ do } \mathcal{C}$	(Iteración)

Sintaxis

$\mathcal{C} ::= [\cdot]$	(Agujero / Hueco)
$\mathcal{C}; \mathcal{C} \mid \mathcal{C}; \mathcal{C}$	(Secuencia)
$\text{if } B \text{ then } \mathcal{C} \text{ else } \mathcal{C}$	(Bifurcación Izquierda)
$\text{if } B \text{ then } \mathcal{C} \text{ else } \mathcal{C}$	(Bifurcación Derecha)
$\text{while } B \text{ do } \mathcal{C}$	(Iteración)

Aplicación de Contexto

$\mathcal{C}[\mathcal{C}]$ denota al comando que se obtiene al reemplazar el agujero $[\cdot]$ en $\mathcal{C}$ por $\mathcal{C}$.

Enfoque clásico: La equivalencia es una congruencia

Congruencia

$\simeq$ es una **congruencia**: Para todo $\mathcal{C}[\cdot]$, se cumple $C_1 \simeq C_2 \implies \mathcal{C}[C_1] \simeq \mathcal{C}[C_2]$

Enfoque clásico: La equivalencia es una congruencia

Congruencia

$\simeq$ es una **congruencia**: Para todo $\mathcal{C}[\cdot]$, se cumple $C_1 \simeq C_2 \implies \mathcal{C}[C_1] \simeq \mathcal{C}[C_2]$

Enfoque clásico: La equivalencia es una congruencia

Congruencia

$\simeq$ es una **congruencia**: Para todo $\mathcal{C}[\cdot]$, se cumple $C_1 \simeq C_2 \implies \mathcal{C}[C_1] \simeq \mathcal{C}[C_2]$

Notar

- Se puede demostrar por inducción en el estructura de $\mathcal{C}$.
- Caso $\mathcal{C} = \mathcal{C}'; C. P; P_1 \simeq P; P_2$ ya que $\llbracket \mathcal{C}[C1] \rrbracket = \llbracket \mathcal{C}'[C1]; C \rrbracket = \llbracket C \rrbracket \circ \llbracket \mathcal{C}'[C1] \rrbracket = \llbracket C \rrbracket \circ \llbracket \mathcal{C}'[C2] \rrbracket = \llbracket \mathcal{C}'[C2]; C \rrbracket = \llbracket \mathcal{C}[C2] \rrbracket$.
- los restantes son análogos

Sistemas actuales de cómputo

- El cómputo puramente secuencial es marginal.

Sistemas actuales de cómputo

- El cómputo puramente secuencial es marginal.
 - **Eficiencia:** Latencia en operaciones de entrada/salida y optimización del uso compartido de CPU.

Sistemas actuales de cómputo

- El cómputo puramente secuencial es marginal.
 - **Eficiencia:** Latencia en operaciones de entrada/salida y optimización del uso compartido de CPU.
 - **Hardware:** Arquitecturas de múltiples núcleos (*multicore*) multinivel.

Sistemas actuales de cómputo

- El cómputo puramente secuencial es marginal.
 - **Eficiencia:** Latencia en operaciones de entrada/salida y optimización del uso compartido de CPU.
 - **Hardware:** Arquitecturas de múltiples núcleos (*multicore*) multinivel.
 - **Distribución:** Componentes de software que se ejecutan concurrentemente en distintos dispositivos físicos.

Orden de ejecución no secuencial

- El orden de ejecución de instrucciones es un **orden parcial**.
- El orden relativo de ejecución entre algunas instrucciones no está especificado:
 - pueden ser ejecutados en cualquier orden

Sintaxis

$x \in \text{Var}$	(Variables)
$n \in \text{Int}$	(Enteros)
$A ::= n \mid x \mid A + A \mid A - A \mid A \times A$	(Expr. Arit.)
$B ::= \text{true} \mid \text{false} \mid A = A \mid A \leq A \mid \neg B \mid B \wedge B$	(Expr. Bool.)
$C ::= \text{skip} \mid x := A \mid C; C \mid \text{if } B \text{ then } C \text{ else } C \mid \text{while } B \text{ do } C$ $\mid C \mid C$	(Comandos)

- Operador $\mid$: paralelo (de menor prioridad)
- $C_1 \mid C_2$: indica el orden relativo de ejecución entre las instrucciones de C_1 y C_2 no está definido.
- Llamamos (de manera imprecisa) **procesos** a C_1 y C_2 .

Ejemplo: Orden Parcial

Definición del escenario

- Sea $C \stackrel{\text{def}}{=} C_1 \mid C_2$ con $C_1 \stackrel{\text{def}}{=} x := 1; y := 2$ y $C_2 \stackrel{\text{def}}{=} z := y$.
-

Ejemplo: Orden Parcial

Definición del escenario

- Sea $C \stackrel{\text{def}}{=} C_1 \mid C_2$ con $C_1 \stackrel{\text{def}}{=} x := 1; y := 2$ y $C_2 \stackrel{\text{def}}{=} z := y$.
 - Las instrucciones en C_1 están totalmente ordenadas: $(x := 1) < (y := 2)$
-

Ejemplo: Orden Parcial

Definición del escenario

- Sea $C \stackrel{\text{def}}{=} C_1 \mid C_2$ con $C_1 \stackrel{\text{def}}{=} x := 1; y := 2$ y $C_2 \stackrel{\text{def}}{=} z := y$.
 - Las instrucciones en C_1 están totalmente ordenadas: $(x := 1) < (y := 2)$
 - Las instrucciones en C_2 están totalmente ordenadas (relación vacía).
-

Ejemplo: Orden Parcial

Definición del escenario

- Sea $C \stackrel{\text{def}}{=} C_1 \mid C_2$ con $C_1 \stackrel{\text{def}}{=} x := 1; y := 2$ y $C_2 \stackrel{\text{def}}{=} z := y$.
- Las instrucciones en C_1 están totalmente ordenadas: $(x := 1) < (y := 2)$
- Las instrucciones en C_2 están totalmente ordenadas (relación vacía).
- Sin embargo, $z := y$ no es comparable con ninguna instrucción de C_1 .

Ejemplo: Orden Parcial

Definición del escenario

- Sea $C \stackrel{\text{def}}{=} C_1 \mid C_2$ con $C_1 \stackrel{\text{def}}{=} x := 1; y := 2$ y $C_2 \stackrel{\text{def}}{=} z := y$.
- Las instrucciones en C_1 están totalmente ordenadas: $(x := 1) < (y := 2)$
- Las instrucciones en C_2 están totalmente ordenadas (relación vacía).
- Sin embargo, $z := y$ no es comparable con ninguna instrucción de C_1 .

Ejemplo: Orden Parcial

Definición del escenario

- Sea $C \stackrel{\text{def}}{=} C_1 \mid C_2$ con $C_1 \stackrel{\text{def}}{=} x := 1; y := 2$ y $C_2 \stackrel{\text{def}}{=} z := y$.
- Las instrucciones en C_1 están totalmente ordenadas: $(x := 1) < (y := 2)$
- Las instrucciones en C_2 están totalmente ordenadas (relación vacía).
- Sin embargo, $z := y$ no es comparable con ninguna instrucción de C_1 .

¿Qué significa esto?

- Una ejecución válida de C debe respetar las restricciones locales:

Ejemplo: Orden Parcial

Definición del escenario

- Sea $C \stackrel{\text{def}}{=} C_1 \mid C_2$ con $C_1 \stackrel{\text{def}}{=} x := 1; y := 2$ y $C_2 \stackrel{\text{def}}{=} z := y$.
- Las instrucciones en C_1 están totalmente ordenadas: $(x := 1) < (y := 2)$
- Las instrucciones en C_2 están totalmente ordenadas (relación vacía).
- Sin embargo, $z := y$ no es comparable con ninguna instrucción de C_1 .

¿Qué significa esto?

- Una ejecución válida de C debe respetar las restricciones locales:
 - Ejecutar $x := 1$ **antes** de $y := 2$.

Ejemplo: Orden Parcial

Definición del escenario

- Sea $C \stackrel{\text{def}}{=} C_1 \mid C_2$ con $C_1 \stackrel{\text{def}}{=} x := 1; y := 2$ y $C_2 \stackrel{\text{def}}{=} z := y$.
- Las instrucciones en C_1 están totalmente ordenadas: $(x := 1) < (y := 2)$
- Las instrucciones en C_2 están totalmente ordenadas (relación vacía).
- Sin embargo, $z := y$ no es comparable con ninguna instrucción de C_1 .

¿Qué significa esto?

- Una ejecución válida de C debe respetar las restricciones locales:
 - Ejecutar $x := 1$ **antes** de $y := 2$.
 - Ejecutar $z := y$ de forma arbitraria (**antes**, **durante** o **después**) de la ejecución de las instrucciones de C_1 .

Solapamiento Temporal

- ¿Pueden las ejecuciones de $x := 1$ y $z := y$ solaparse en el mismo instante de tiempo físico?

^aTaxonomía de Flynn (1966).

Solapamiento Temporal

- ¿Pueden las ejecuciones de $x := 1$ y $z := y$ solaparse en el mismo instante de tiempo físico?
- Depende de la arquitectura de hardware.^a

^aTaxonomía de Flynn (1966).

Solapamiento Temporal

- ¿Pueden las ejecuciones de $x := 1$ y $z := y$ solaparse en el mismo instante de tiempo físico?
- Depende de la arquitectura de hardware.^a
 - **Imposible:** Si disponemos de un único procesador que ejecuta una sola instrucción a la vez (**SISD**).

^aTaxonomía de Flynn (1966).

Solapamiento Temporal

- ¿Pueden las ejecuciones de $x := 1$ y $z := y$ solaparse en el mismo instante de tiempo físico?
- Depende de la arquitectura de hardware.^a
 - **Imposible:** Si disponemos de un único procesador que ejecuta una sola instrucción a la vez (**SISD**).
 - **Posible:** Si el sistema cuenta con múltiples procesadores, ya sean núcleos o distribuidos (**MIMD**).

^aTaxonomía de Flynn (1966).

Paralelismo

Al menos dos pasos de la ejecución de un programa se solapan en el tiempo (requiere hardware multiprocesador).

Paralelismo y Concurrency

Paralelismo

Al menos dos pasos de la ejecución de un programa se solapan en el tiempo (requiere hardware multiprocesador).

Concurrency

Paralelismo **potencial**.

Un único procesador ejecutará de manera secuencial alguna **linearización del orden parcial**.

Paralelismo y Concurrency

Paralelismo

Al menos dos pasos de la ejecución de un programa se solapan en el tiempo (requiere hardware multiprocesador).

Concurrencia

Paralelismo **potencial**.

Un único procesador ejecutará de manera secuencial alguna **linearización del orden parcial**.

Importante

Ni la ejecución paralela ni la concurrente se explican con un orden total.

¿Por qué concurrencia?

Concurrencia como Abstracción

Una abstracción que permite comprender la ejecución de programas que comparten recursos:

- Evita considerar detalles específicos de su ejecución física.

¿Por qué concurrencia?

Concurrencia como Abstracción

Una abstracción que permite comprender la ejecución de programas que comparten recursos:

- Evita considerar detalles específicos de su ejecución física.
- En particular, independiza el modelo matemático de la cantidad de procesadores reales sobre los que se ejecuta el software.

¿Cómo se estudia la concurrencia?

Modelos Semánticos

1. **Concurrencia Real (*True Concurrency*):** Modelos basados puramente en **órdenes parciales**.

En esta materia:

- Utilizaremos mayormente el entrelazado porque es
 - el más comunmente aceptado... especialmente en la literatura tradicional.
 - “el más simple”.

¿Cómo se estudia la concurrencia?

Modelos Semánticos

1. **Concurrencia Real (*True Concurrency*)**: Modelos basados puramente en **órdenes parciales**.
2. **Entrelazado (*Interleaving*)**: Modelos que consideran la abstracción de un único procesador virtual.

En esta materia:

- Utilizaremos mayormente el entrelazado porque es
 - el más comunmente aceptado... especialmente en la literatura tradicional.
 - “el más simple”.

¿Cómo se estudia la concurrencia?

Modelos Semánticos

1. **Concurrencia Real (*True Concurrency*)**: Modelos basados puramente en **órdenes parciales**.
2. **Entrelazado (*Interleaving*)**: Modelos que consideran la abstracción de un único procesador virtual.
 - La semántica de un programa captura el conjunto de todos los posibles órdenes totales que constituyen las **linearizaciones del orden parcial**.

En esta materia:

- Utilizaremos mayormente el entrelazado porque es
 - el más comunmente aceptado... especialmente en la literatura tradicional.
 - “el más simple”.

Ejecución concurrente

Ejecución concurrente

La ejecución de un programa concurrente consiste en **ejecutar un entrelazamiento arbitrario** de las instrucciones atómicas de sus procesos.

Ejecución de $C \stackrel{\text{def}}{=} x := 1; y := 2 \mid z := y$

- Asumiendo cada asignación como atómica, se puede ejecutar como:
 - $z := y; x := 1; y := 2$, o
 - $x := 1; z := y; y := 2$, o
 - $x := 1; y := 2; z := y$.
- La **elección es arbitraria**, no hay garantías sobre cuál será elegida.

Abstracción del tiempo

Se **ignoran cuestiones de tiempo** para analizar un programa:

- La próxima instrucción a ejecutar puede pertenecer a **cualquier proceso**.
- La correctitud de un programa concurrente **no depende** de suposiciones sobre los tiempos exactos de ejecución.

Discretización del espacio de estados

Al eliminar la variable temporal, el análisis se simplifica:

- Sólo se debe considerar una cantidad **finita o enumerable** de secuencias posibles de ejecución.

No determinismo

Ejecución de $C \stackrel{\text{def}}{=} x := 1; y := 2 \mid z := y$

- Asumiendo cada asignación como atómica, se puede ejecutar como:
 - $z := y; x := 1; y := 2:$ $\lambda\sigma.\sigma[x \mapsto 1, y \mapsto 2, z \mapsto \sigma y]^a$
 - $x := 1; z := y; y := 2:$ $\lambda\sigma.\sigma[x \mapsto 1, y \mapsto 2, z \mapsto \sigma y]$
 - $x := 1; y := 2; z := y:$ $\lambda\sigma.\sigma[x \mapsto 1, y \mapsto 2, z \mapsto 2]$
- Los dos primeros producen el mismo resultado, pero el **tercero** es distinto.

^aDenota actualización de estado

No determinismo

Distintas ejecuciones de un mismo programa con un mismo estado inicial pueden arrojar **resultados distintos**.

Hacia una semántica denotacional

Notación

Usaremos el operador $\oplus$ para denotar no determinismo

$$\lambda\sigma.\sigma_1 \oplus \sigma_2 \oplus \cdots \oplus \sigma_n$$

la función puede producir cualquier σ_i ($1 \leq i \leq n$)

$$C \stackrel{\text{def}}{=} x := 1; y := 2 \mid z := y$$

$$\llbracket C \rrbracket = \lambda\sigma.\sigma[x \mapsto 1, y \mapsto 2, z \mapsto \sigma y] \oplus \sigma[x \mapsto 1, y \mapsto 2, z \mapsto 2]$$

Limitaciones del enfoque anterior

No composicionalidad

- Sean $C_1 \stackrel{\text{def}}{=} x := 1$ y $C_2 \stackrel{\text{def}}{=} x := 0; x := x + 1$.
- Luego (asumiendo que cada asignación es atómica):
 $\llbracket C_1 \rrbracket =$

Limitación

Esta visión pierde el hecho de que el estado puede ser alterado externamente por otro proceso durante la ejecución.

Limitaciones del enfoque anterior

No composicionalidad

- Sean $C_1 \stackrel{\text{def}}{=} x := 1$ y $C_2 \stackrel{\text{def}}{=} x := 0; x := x + 1$.
- Luego (asumiendo que cada asignación es atómica):
 $\llbracket C_1 \rrbracket = \lambda \sigma. \sigma[x \mapsto 1]$.
 $\llbracket C_2 \rrbracket =$

Limitación

Esta visión pierde el hecho de que el estado puede ser alterado externamente por otro proceso durante la ejecución.

Limitaciones del enfoque anterior

No composicionalidad

- Sean $C_1 \stackrel{\text{def}}{=} x := 1$ y $C_2 \stackrel{\text{def}}{=} x := 0; x := x + 1$.
- Luego (asumiendo que cada asignación es atómica):
$$\llbracket C_1 \rrbracket = \lambda \sigma. \sigma[x \mapsto 1]. \quad \llbracket C_1 | C_1 \rrbracket =$$
$$\llbracket C_2 \rrbracket = \lambda \sigma. \sigma[x \mapsto 1].$$

Limitación

Esta visión pierde el hecho de que el estado puede ser alterado externamente por otro proceso durante la ejecución.

Limitaciones del enfoque anterior

No composicionalidad

- Sean $C_1 \stackrel{\text{def}}{=} x := 1$ y $C_2 \stackrel{\text{def}}{=} x := 0; x := x + 1$.
- Luego (asumiendo que cada asignación es atómica):
 $\llbracket C_1 \rrbracket = \lambda\sigma.\sigma[x \mapsto 1].$ $\llbracket C_1 | C_1 \rrbracket = \lambda\sigma.\sigma[x \mapsto 1].$
 $\llbracket C_2 \rrbracket = \lambda\sigma.\sigma[x \mapsto 1].$ $\llbracket C_1 | C_2 \rrbracket =$

Limitación

Esta visión pierde el hecho de que el estado puede ser alterado externamente por otro proceso durante la ejecución.

Limitaciones del enfoque anterior

No composicionalidad

- Sean $C_1 \stackrel{\text{def}}{=} x := 1$ y $C_2 \stackrel{\text{def}}{=} x := 0; x := x + 1$.
- Luego (asumiendo que cada asignación es atómica):
$$\llbracket C_1 \rrbracket = \lambda\sigma.\sigma[x \mapsto 1].$$
$$\llbracket C_2 \rrbracket = \lambda\sigma.\sigma[x \mapsto 1].$$
$$\llbracket C_1 | C_1 \rrbracket = \lambda\sigma.\sigma[x \mapsto 1].$$
$$\llbracket C_1 | C_2 \rrbracket = \lambda\sigma.\sigma[x \mapsto 1] \oplus \sigma[x \mapsto 2].$$
- **Problema:**

Limitación

Esta visión pierde el hecho de que el estado puede ser alterado externamente por otro proceso durante la ejecución.

No composicionalidad

- Sean $C_1 \stackrel{\text{def}}{=} x := 1$ y $C_2 \stackrel{\text{def}}{=} x := 0; x := x + 1$.
- Luego (asumiendo que cada asignación es atómica):
$$\llbracket C_1 \rrbracket = \lambda\sigma.\sigma[x \mapsto 1]. \quad \llbracket C_1 | C_1 \rrbracket = \lambda\sigma.\sigma[x \mapsto 1].$$
$$\llbracket C_2 \rrbracket = \lambda\sigma.\sigma[x \mapsto 1]. \quad \llbracket C_1 | C_2 \rrbracket = \lambda\sigma.\sigma[x \mapsto 1] \oplus \sigma[x \mapsto 2].$$
- **Problema:** La *equivalencia* no es una congruencia:
 - $\exists \mathcal{C} = [\cdot] \mid C_1$ tal que $C_1 \simeq C_2$ y $\mathcal{C}[C_1] \not\approx \mathcal{C}[C_2]$.

Limitación

Limitaciones del enfoque anterior

No composicionalidad

- Sean $C_1 \stackrel{\text{def}}{=} x := 1$ y $C_2 \stackrel{\text{def}}{=} x := 0; x := x + 1$.
- Luego (asumiendo que cada asignación es atómica):
$$\llbracket C_1 \rrbracket = \lambda\sigma.\sigma[x \mapsto 1]. \quad \llbracket C_1 | C_1 \rrbracket = \lambda\sigma.\sigma[x \mapsto 1].$$
$$\llbracket C_2 \rrbracket = \lambda\sigma.\sigma[x \mapsto 1]. \quad \llbracket C_1 | C_2 \rrbracket = \lambda\sigma.\sigma[x \mapsto 1] \oplus \sigma[x \mapsto 2].$$
- **Problema:** La *equivalencia* no es una congruencia:
 - $\exists \mathcal{C} = [\cdot] \mid C_1$ tal que $C_1 \simeq C_2$ y $\mathcal{C}[C_1] \not\approx \mathcal{C}[C_2]$.

Limitación

Esta visión pierde el hecho de que el estado puede ser alterado externamente por otro proceso durante la ejecución.

Mismo resultado, diferente comportamiento

Sean los procesos de bucle infinito:

- $C_1 \stackrel{\text{def}}{=} \text{while true do skip}$
- $C_2 \stackrel{\text{def}}{=} \text{while true do } x := x + 1$
- $C_3 \stackrel{\text{def}}{=} \text{while true do } y := y + 1$

Aislados, la denotación de todos ellos es idéntica (divergencia pura):

$$\llbracket C_1 \rrbracket = \llbracket C_2 \rrbracket = \llbracket C_3 \rrbracket = \lambda\sigma. \perp$$

Mismo resultado, diferente comportamiento

Sean los procesos de bucle infinito:

- $C_1 \stackrel{\text{def}}{=} \text{while true do skip}$
- $C_2 \stackrel{\text{def}}{=} \text{while true do } x := x + 1$
- $C_3 \stackrel{\text{def}}{=} \text{while true do } y := y + 1$

Aislados, la denotación de todos ellos es idéntica (divergencia pura):

$$\llbracket C_1 \rrbracket = \llbracket C_2 \rrbracket = \llbracket C_3 \rrbracket = \lambda\sigma. \perp$$

Sin embargo, tienen **distinta interacción**:

- C_1 es **independiente** (no altera variables).
- C_2 **comunica o interactúa** modificando continuamente la variable x .
- C_3 **comunica o interactúa** modificando continuamente la variable y .

Mecanismo de interacción

La definición de una semántica composicional debe considerar la manera en que los procesos comunican o interactúan.

Importante

Desarrollar una semántica denotacional (composicional) para lenguajes concurrentes está fuera del alcance de este curso.

Para interesados:

- **Resumptions:** Hennessy, M. y Plotkin, G. D. (1979). *Full abstraction for a simple parallel programming language*.
- **Trace Semantics:** Brookes, S. (1996). *Full abstraction for a shared-variable parallel language*.

Modelos de interacción: Categorías principales

- **Memoria compartida:**
 - Los procesos leen y escriben en un espacio de direcciones común.
 - Requiere protocolos o primitivas para coordinar el acceso a la memoria.
- **Intercambio de mensajes:**
 - Los procesos tienen memorias aisladas y no comparten datos directamente.
 - Comunican explícitamente mediante operaciones de envío y recepción.
- Presentan muchas variantes.

Características salientes

- No determinismo intrínseco.
- El resultado final no siempre es interesante (importan los programas que no terminan).
- Interacción/comunicación.

Atomicidad

Una instrucción es atómica si se puede ejecutar completamente sin la posibilidad de ser intercalada con la ejecución de otra instrucción atómica.

Ejecución simultánea:

- Si dos instrucciones atómicas se ejecutan “al mismo tiempo”, su resultado equivale a la ejecución secuencial en algún orden arbitrario.
- No existen estados intermedios visibles de la ejecución de una acción atómica.

Atomicidad: Alto vs. Bajo nivel

- Al definir qué acciones son **atómicas**, se deben considerar las garantías de ejecución del procesador.
- En general, vamos a asumir que las **lecturas y escrituras de variables compartidas** son atómicas, así como también la **evaluación de expresiones sobre variables locales (registros)**.
- La asignación $x := x + 1$ se descompone como la **secuencia de acciones atómicas** (x' es una variable local/no compartida):
 1. $x' := x$ (Lectura de la variable compartida)
 2. $x' := x' + 1$ (Modificación local interna)
 3. $x := x'$ (Escritura en la variable compartida)

Atomicidad: Alto vs. Bajo nivel

- Al definir qué acciones son **atómicas**, se deben considerar las garantías de ejecución del procesador.
- En general, vamos a asumir que las **lecturas y escrituras de variables compartidas** son atómicas, así como también la **evaluación de expresiones sobre variables locales (registros)**.
- La asignación $x := x + 1$ se descompone como la **secuencia de acciones atómicas** (x' es una variable local/no compartida):
 1. $x' := x$ (Lectura de la variable compartida)
 2. $x' := x' + 1$ (Modificación local interna)
 3. $x := x'$ (Escritura en la variable compartida)
- Cada una de estas tres (sub)instrucciones es por sí misma **atómica**.

Ejemplo

- Sea $C \stackrel{\text{def}}{=} x := x + 1$.
- Asumiendo que la asignación es atómica:

$$\llbracket C \mid C \rrbracket =$$

Ejemplo

- Sea $C \stackrel{\text{def}}{=} x := x + 1$.
- Asumiendo que la asignación es atómica:

$$\llbracket C \mid C \rrbracket = \lambda\sigma.\sigma[x \mapsto \sigma x + 2]$$

- Reescribiendo con x' local: $C \stackrel{\text{def}}{=} x' := x; x' := x' + 1; x := x'$:
 - Lectura y escritura atómica de variables compartidas:

$$\llbracket C \mid C \rrbracket =$$

Ejemplo

- Sea $C \stackrel{\text{def}}{=} x := x + 1$.
- Asumiendo que la asignación es atómica:

$$\llbracket C \mid C \rrbracket = \lambda\sigma.\sigma[x \mapsto \sigma x + 2]$$

- Reescribiendo con x' local: $C \stackrel{\text{def}}{=} x' := x; x' := x' + 1; x := x'$:
 - Lectura y escritura atómica de variables compartidas:

$$\llbracket C \mid C \rrbracket = \lambda\sigma.\sigma[x \mapsto \sigma x + 1, \dots] \oplus \sigma[x \mapsto \sigma x + 2, \dots]$$

- Se pierde un incremento debido al entrelazamiento.
- En general, no nos van a interesar los cambios en las variables locales (x').

Independencia del tiempo

Principio fundamental

La semántica y la correctitud de un programa concurrente son **independientes** de los tiempos de ejecución.

Diseño de programas

- **Sin suposiciones temporales:** Ningún programa concurrente puede basar su lógica en el paso del tiempo o velocidad de la CPU.
- **Indeterminación de velocidad:** Una instrucción puede tardar microsegundos o días en ejecutar; el programa debe ser válido en cualquier escenario.
- **Sincronización explícita:** Si el orden de ejecución importa, el programa lo debe garantizar independiente del paso del tiempo.

Ejemplo

- Considerar
 - $C_1 \stackrel{\text{def}}{=} \text{sleep}(1000); x := 1$
 - $C_2 \stackrel{\text{def}}{=} x := 2$
- Intuitivamente, se podría pensar que C_2 terminará antes.
- Bajo el principio de independencia del tiempo resulta:

$$\llbracket C_1 \mid C_2 \rrbracket = \lambda \sigma. \sigma[x \mapsto 1] \oplus \sigma[x \mapsto 2]$$

Contenidos

- Memoria compartida:
 - Exclusión Mutua, Algoritmos de exclusion mutua

Contenidos

- Memoria compartida:
 - Exclusión Mutua, Algoritmos de exclusion mutua (Dekker & Bakery).

Contenidos

- Memoria compartida:
 - Exclusión Mutua, Algoritmos de exclusion mutua (Dekker & Bakery).
Semáforos y Monitores.
 - Problemas clásicos:

Contenidos

- Memoria compartida:
 - Exclusión Mutua, Algoritmos de exclusion mutua (Dekker & Bakery). Semáforos y Monitores.
 - Problemas clásicos: Filósofos comensales, Lectores/Escritores, ...

Contenidos

- Memoria compartida:
 - Exclusión Mutua, Algoritmos de exclusion mutua (Dekker & Bakery). Semáforos y Monitores.
 - Problemas clásicos: Filósofos comensales, Lectores/Escritores, ...
 - Estructuras de datos concurrentes y programación libre de locks. Software transaction memories (STM)

Contenidos

- Memoria compartida:
 - Exclusión Mutua, Algoritmos de exclusion mutua (Dekker & Bakery). Semáforos y Monitores.
 - Problemas clásicos: Filósofos comensales, Lectores/Escritores, ...
 - Estructuras de datos concurrentes y programación libre de locks. Software transaction memories (STM)
- Intercambio de Mensajes:
 - Comunicación sincrónica
 - Comunicación asincrónica (continuaciones)

Contenidos

- Memoria compartida:
 - Exclusión Mutua, Algoritmos de exclusion mutua (Dekker & Bakery). Semáforos y Monitores.
 - Problemas clásicos: Filósofos comensales, Lectores/Escritores, ...
 - Estructuras de datos concurrentes y programación libre de locks. Software transaction memories (STM)
- Intercambio de Mensajes:
 - Comunicación sincrónica
 - Comunicación asincrónica (continuaciones)
- Modelo de actores.
- Modelos formales de concurrencia (LTS y LTL/CTL).

Pseudocódigo para algoritmos concurrentes

- Usaremos un pseudocódigo estructurado para describir nuestros algoritmos.
- Las palabras clave:
 - `global` : indica que una variable es compartida.
 - `thread` : indica que un flujo de ejecución es concurrente.

Programa con variable compartida

```
1  global x
2
3  thread T1
4  x = 0
5
6  thread T2
7  x = 1
```

- T1 y T2 son flujos que ejecutan concurrentemente
- Comparten a la variable `x`

Pseudocódigo para algoritmos concurrentes

- Es posible declarar variables locales (sin cualificador)

Programa con variable compartida

```
1  global x = 0
2
3  thread T1
4    temp
5    temp = x
6    x = temp + 1
7
8  thread T2
9    temp
10   temp = x
11   x = temp + 1
12
```

- T1 y T2 declaran la variable local `temp`.
- Son dos variable homónimas.
- Asumimos que podemos aplicar α -renombre en variables locales.

Pseudocódigo para algoritmos concurrentes

- Acciones atómicas: vamos a etiquetar instrucciones cuando cada línea de ejecución es atómica.

Programa con variable compartida

```
1  global x = 0
2
3  thread p
4  temp
5  p1:   temp = x
6  p2:   x = temp + 1
7
8  thread q
9  temp
10 q1:   temp = x
11 q2:   x = temp + 1
```

- p1, p2, q1 y q2 son etiquetas y cada instrucción es atómica.

Modelo de ejecución de un programa

- **Todas** las ejecuciones de un programa de N procesos se describe como un grafo dirigido (posiblemente infinito):
 - Estados (nodos):
 - N etiquetas de próxima instrucción, una para cada programa.
 - la asignación de valores a variables globales y locales
 - Transición (arcos): Hay una transición entre s_1 y s_2 si s_2 se obtiene ejecutando una de las próximas acciones de s_1
- **Una** ejecución es un camino (posiblemente infinito) en el grafo.

Modelo de Ejecución

Considerar los threads

```
1 global x = 0
2
3 thread p
4 p1:    x = x + 1
5
6 thread q
7 q1:    x = x + 1
```

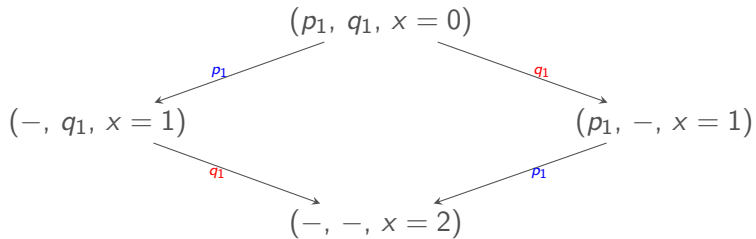
Modelo de Ejecución

Considerar los threads

```
1 global x = 0
2
3 thread p
4 p1:    x = x + 1
5
6 thread q
7 q1:    x = x + 1
```

¿Cuál es el diagrama de transición de estados?

Grafo de Transición - Asignación Atómica Global

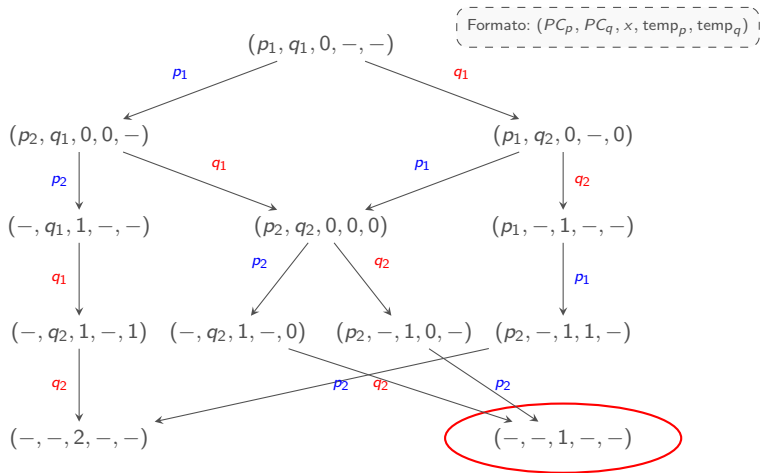


Modelo de Ejecución

```
1 global x = 0
2
3 thread p
4 temp
5 p1:    temp = x
6 p2:    x = temp + 1
7
8 thread q
9 temp
10 q1:    temp = x
11 q2:    x = temp + 1
```

¿Cuál es el diagrama de transición de estados?

Grafo de Transición - Asignación No Atómica (Data Race)



Pérdida de actualización

Correctitud de programas concurrentes

Correctitud

- No se analiza en términos de calcular un resultado funcional único o estado final.
- Porque hay programas que no terminan.
- Se usan **propiedades de la computación**:
 - *Safety*
 - *Liveness*

Propiedad de *Safety*

- Establece que algo malo **nunca** ocurre.
- Debe valer en **todo estado de cómputo**.

Ejemplo

- En un sistema operativo: *“El cursor del mouse siempre se muestra en pantalla”*.
- Si se vale esta propiedad, el cursor jamás desaparecerá, independientemente de qué programas se estén ejecutando.

Propiedades de Liveness

Propiedad de *Liveness*

- Establece que algo bueno **tarde o temprano** ocurrirá.
- Vale, si para todo estado **cualquier estado** alcanzable siempre es posible alcanzar (en 0 o mas pasos) un estado que cumpla la propiedad.

Ejemplo

- En un sistema operativo: *“Si haces clic en el botón del mouse, tarde o temprano el cursor del mouse cambia de forma”*.
- El sistema puede no responder inmediatamente, pero no puede postergar la respuesta indefinidamente.

Especificación de propiedades de Safety

- Las propiedades de *safety* suelen tomar la forma de: **Siempre, algo “malo” no es verdadero.**
- Estas propiedades se satisfacen trivialmente con un programa que no hace nada.

Desafío

Escribir programas que realicen tareas útiles (satisfacen propiedades de *liveness* sin violar las propiedades de *safety*).

- Un sistema que solo muestra el cursor del mouse inmóvil es seguro (*safe*), pero completamente inútil por falta de progreso (*liveness*).

Definición de Correctitud Concurrente

Correctitud de un programa concurrente

Un programa concurrente es **correcto** si y sólo si satisface simultáneamente tanto sus propiedades de ***safety*** como sus propiedades de ***liveness***.

Fairness

- La correctitud exige considerar todas las ejecuciones.
- ¿Tiene sentido escenarios donde las instrucciones de un proceso específico **jamás** se ejecutan..

Ejemplo

```
1      global n = 0
2      global flag = false
3
4      thread p
5      p1:  while flag = false
6      p2:  n = 1 - n
7
8      thread q
9      q1:  flag = true
10
```

Fairness

- La correctitud exige considerar todas las ejecuciones.
- ¿Tiene sentido escenarios donde las instrucciones de un proceso específico **jamás** se ejecutan..

Ejemplo

```
1      global n = 0
2      global flag = false
3
4      thread p
5      p1:  while flag = false
6      p2:  n = 1 - n
7
8      thread q
9      q1:  flag = true
10
```

Definición de Continuamente Habilitada en Ben-Ari

Instrucción habilitada

Una instrucción p_j de un proceso p está **habilitada** en un estado s si el puntero de instrucción del proceso apunta a p_j y **puede ser ejecutada**.

- Las asignaciones (ej. $x := 1$) y las sentencias de control no pueden ser bloqueadas por el entorno.
- Una vez que el puntero de control llega a esa instrucción, la instrucción permanece en estado habilitado (*enabled*) hasta que es ejecutada.
- Nada que haga otro proceso la puede “deshabilitar”.

Ejecución (débilmente) fair

- Una ejecución es **(débilmente) fair** si una instrucción que está **continuamente habilitada** tarde o temprano se ejecuta.
- Dada una ejecución $s_0, s_1, \dots, s_i$, si p_k está continuamente habilitada en s_i , entonces p_k es ejecutada en un estado posterior s_k para algún $k > i$.

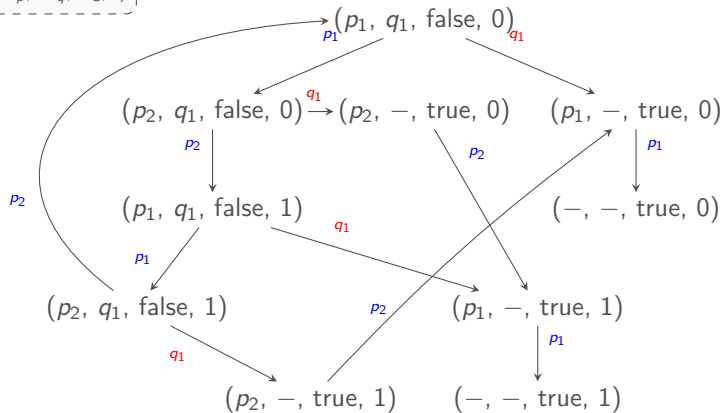
Ejemplo

```
1      global n = 0
2      global flag = false
3
4      thread p
5      p1:  while flag = false
6      p2:      n = 1 - n
7
8      thread q
9      q1:  flag = true
10
```

- El programa termina en ejecuciones fair.
- Toda ejecución fair debe incluir q_1 .
- Esto garantiza que q termine.
- Luego p termina (al máximo en dos pasos)

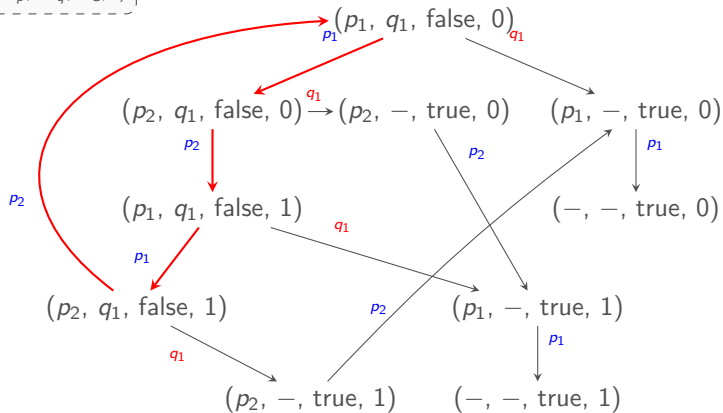
Grafo de Transición

Formato: $(PC_p, PC_q, \text{flag}, n)$



Grafo de Transición - Computación infinita (no terminación)

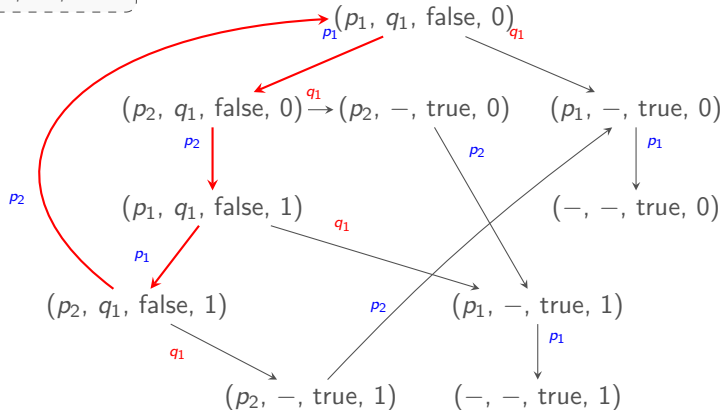
Formato: $(PC_p, PC_q, \text{flag}, n)$



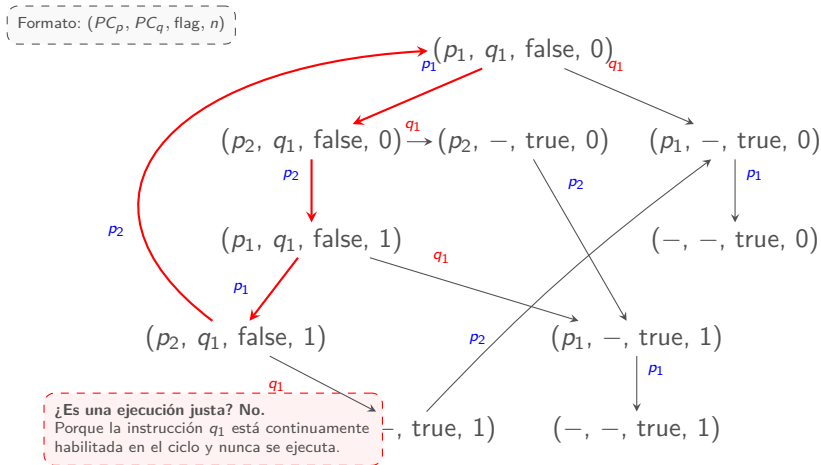
c

Grafo de Transición - Computación infinita (no terminación)

Formato: $(PC_p, PC_q, \text{flag}, n)$



Grafo de Transición - Computación infinita (no terminación)



Sección crítica

Llamamos sección crítica a una parte del programa que no puede ser ejecutada concurrentemente con otra sección crítica del mismo programa.

Sección crítica

Llamamos sección crítica a una parte del programa que no puede ser ejecutada concurrentemente con otra sección crítica del mismo programa.

Exclusión mutua

Llamamos exclusión mutua al problema de asegurar que dos (o mas) threads no ejecutan simultáneamente su sección crítica.

Esquema general

Existen N procesos que tienen la siguiente estructura

```
1 shared variables
2
3 thread id = i
4 while(true){
5     seccion no critica
6     pre-protocol
7     seccion critica
8     post-protocol
9 }
```

- No hay variables compartidas entre sección crítica y no crítica.
- La sección crítica siempre termina.
- La no crítica no necesariamente termina.

Requerimientos de la exclusión mutua

1. **Exclusión Mutua (Safety):** En cualquier momento hay como máximo un proceso en la región crítica.

Requerimientos de la exclusión mutua

1. **Exclusión Mutua (Safety):** En cualquier momento hay como máximo un proceso en la región crítica.
2. **Ausencia de *deadlocks* (Liveness):** Si varios procesos intentan entrar a la sección crítica tarde o temprano alguno entra.

Requerimientos de la exclusión mutua

1. **Exclusión Mutua (Safety):** En cualquier momento hay como máximo un proceso en la región crítica.
2. **Ausencia de *deadlocks* (Liveness):** Si varios procesos intentan entrar a la sección crítica tarde o temprano alguno entra.
3. **Ausencia de inanición (garantía de entrada) (Liveness):** Cualquier proceso que intenta entrar a su sección crítica tarde o temprano entra.

Exclusión mutua

Problema

¿Podemos resolver el problema de la exclusión mutua para dos procesos asumiendo que las únicas operaciones atómicas son la lectura y la escritura de variables?

Algoritmo I (levanto la bandera)

Algoritmo I (levanto la bandera)

```
shared flag = false
```

```
thread p
p0: while(true)
p1:   seccion no critica
p2:   while (flag);
p3:   flag = true
p4:   seccion critica
p5:   flag = false
```

```
thread q
q0: while(true)
q1:   seccion no critica
q2:   while (flag);
q3:   flag = true
q4:   seccion critica
q5:   flag = false
```

- ¿Cómo lo analizamos?
- Razonamos sobre el modelo de cómputo
- Problema: El tamaño (explosión combinatorial).
- Vamos a simplificar (para razonar manualmente)

Simplificación para análisis

- Es riesgos, pero vamos a considerar la secciones críticas y no críticas como comentarios
- Vamos a obviar el while.

Simplificación para análisis

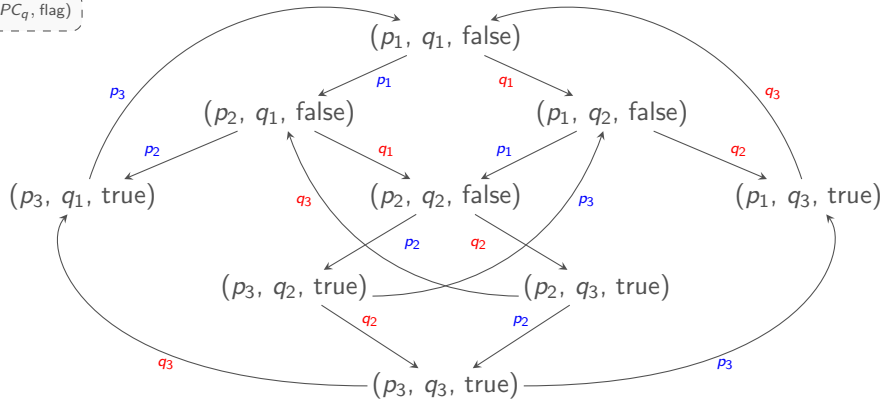
- Es riesgos, pero vamos a considerar la secciones críticas y no críticas como comentarios
- Vamos a obviar el while.

```
1             shared flag = false
2
3     thread p
4     while (true)
5 p1:     while (flag)
6 p2:     flag = true
7 p3:     flag = false
```

```
thread q
while(true)
q1:     while (flag)
q2:     flag = true
q3:     flag = false
```

Modelo de cómputo (simplificado)

Formato: $(PC_p, PC_q, \text{flag})$



Análisis

```
1         shared flag = false
2
3     thread p           thread q
4     while (true)       while(true)
5 p1:   while (flag)     q1:   while (flag)
6 p2:   flag = true      q2:   flag = true
7 p3:   flag = false     q3:   flag = false
```

- **Exclusión mutua (Safety):** En cualquier momento hay como máximo un proceso en la región crítica.
- ¿Cuándo un proceso está en la sección crítica?

```
1          shared flag = false
2
3      thread p          thread q
4      while (true)      while(true)
5  p1:  while (flag)      q1:  while (flag)
6  p2:  flag = true        q2:  flag = true
7  p3:  flag = false       q3:  flag = false
```

- **Exclusión mutua (Safety):** En cualquier momento hay como máximo un proceso en la región crítica.
- ¿Cuándo un proceso está en la sección crítica? Si p está en p_3 o q está en q_3 .
- ¿Cuál es un estado de violación de exclusión mutua?

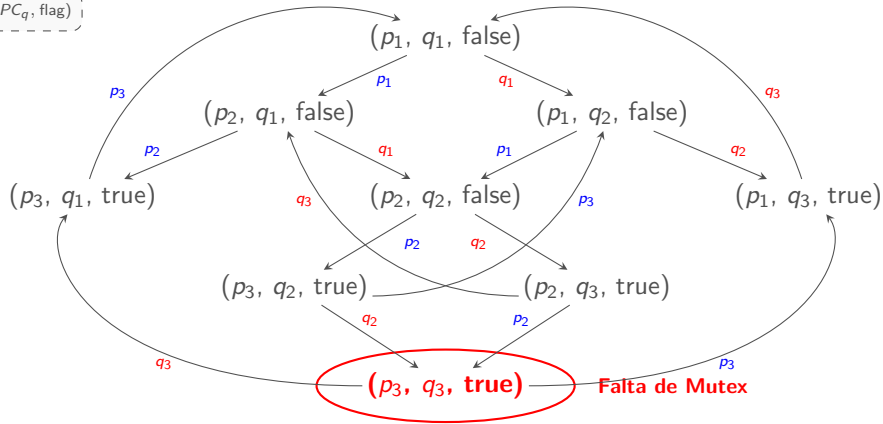
Análisis

```
1          shared flag = false
2
3      thread p          thread q
4      while (true)      while(true)
5 p1:   while (flag)      q1:   while (flag)
6 p2:   flag = true       q2:   flag = true
7 p3:   flag = false      q3:   flag = false
```

- **Exclusión mutua (Safety):** En cualquier momento hay como máximo un proceso en la región crítica.
- ¿Cuándo un proceso está en la sección crítica? Si p está en $p3$ o q está en $q3$.
- ¿Cuál es un estado de violación de exclusión mutua? Un estado $(p3, q3, ?)$ donde ambos procesos coexisten en su sección crítica.

Modelo de cómputo (simplificado)

Formato: $(PC_p, PC_q, \text{flag})$



Análisis

```
shared flag = false
```

```
thread p
while (true)
p1:   while (flag)
p2:   flag = true
p3:   flag = false
```

```
thread q
while(true)
q1:   while (flag)
q2:   flag = true
q3:   flag = false
```

Algoritmo II (levanto mi bandera)

```
1
2         global flag = {false, false}
3
4
5  thread id = 0
6  while(true){
7      // seccion no critica
8      otro = (id + 1) % 2
9      flag[id] = true
10     while (flag[otro]);
11     // seccion critica
12     flag[id] = false
13 }

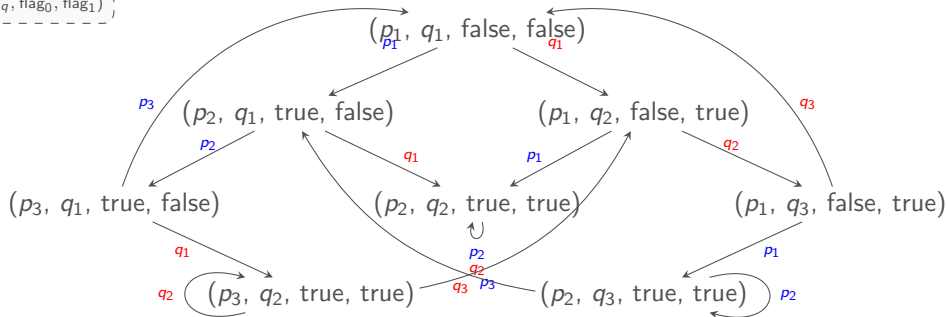
        thread id = 1
        while(true){
            // seccion no critica
            otro = (id + 1) % 2
            flag[id] = true
            while (flag[otro]);
            // seccion critica
            flag[id] = false
        }
```

Algoritmo II (levanto mi bandera)

```
1
2         global flag = {false, false}
3
4     thread                                thread
5     while(true)                          while(true)
6 p1:  flag[0] = true                      q1:  flag[1] = true
7 p2:  while (flag[1]);                    q2:  while (flag[0]);
8 p3:  flag[0] = false                     q3:  flag[id] = false
```

Segundo Intento: Modelo de cómputo

Formato: $(PC_p, PC_q, \text{flag}_0, \text{flag}_1)$

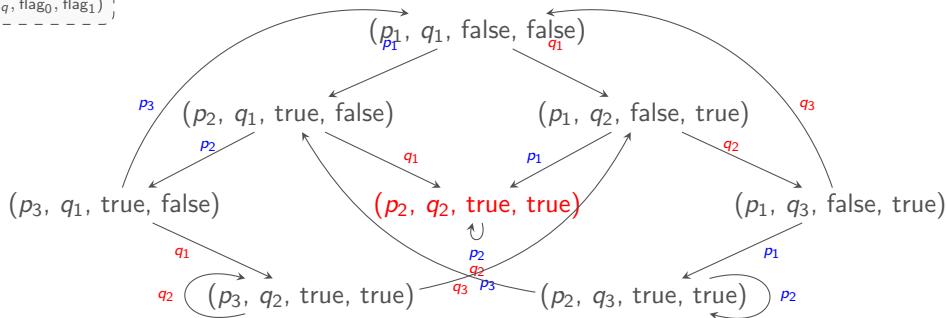


Algoritmo II (levanto mi bandera)

```
1
2         global flag = {false, false}
3
4     thread p                                thread q
5     while(true)                            while(true)
6 p1:  flag[0] = true                        q1:  flag[1] = true
7 p2:  while (flag[1]);                      q2:  while (flag[0]);
8 p3:  flag[0] = false                       q3:  flag[id] = false
```

Segundo Intento: Modelo de cómputo

Formato: $(PC_p, PC_q, \text{flag}_0, \text{flag}_1)$



Nota: Técnicamente representa un **livelock**. Es un estado donde los hilos continúan ejecutando instrucciones activamente (espera activa), pero el sistema global no logra avanzar.

Algoritmo III (uso un turno)

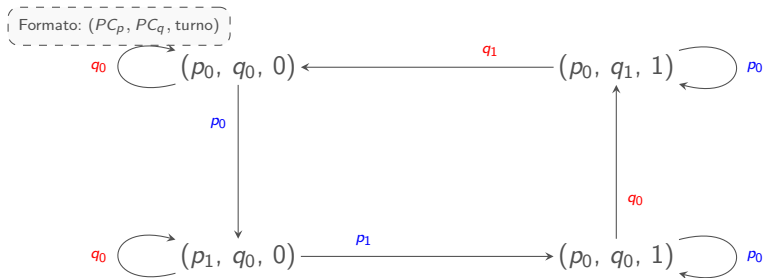
```
1
2         global turno = 0
3
4
5  thread id = 0
6  while(true){
7      // seccion no critica
8      while (turno != id);
9      // seccion critica
10     turno = (id + 1) % 2
11     // seccion no critica
12 }

        thread id = 1
        while(true){
            // seccion no critica
            while (turno != id);
            // seccion critica
            turno = (id + 1) % 2
            // seccion no critica
        }
```

Algoritmo III (Simplificación)

```
1
2         global turno = 0
3
4  thread p                                thread q
5  while(true){                            while(true){
6  p0:  while (turno != 0);                q0:  while (turno != 1);
7  p1:  turno = 1                          q1:  turno = 0
```

Primer Intento: Modelo de cómputo



Algoritmo III (Simplificación)

```
1
2         global turno = 0
3
4  thread p                thread q
5  while(true){            while(true){
6  p0:  while (turno != 0);  q0:  while (turno != 1);
7  p1:  turno = 1           q1:  turno = 0
```

Algoritmo III (Simplificación)

```
1
2             global turno = 0
3
4  thread p           thread q
5  while(true){      while(true){
6  p0:  while (turno != 0);    q0:  while (turno != 1);
7  p1:  turno = 1             q1:  turno = 0
```

- **Mutex:**
- **Ausencia de deadlocks:**
- **Ausencia de inanición:**
 - Nuestra simplificación, tiene un problema.
 - Eliminó las secciones no críticas, que pueden no terminar.

Algoritmo III (Simplificación)

```
1
2             global turno = 0
3
4  thread p           thread q
5  while(true){       while(true){
6  p0:  while (turno != 0);    q0:  while (turno != 1);
7  p1:  turno = 1             q1:  turno = 0
```

- **Mutex:** Sí
- **Ausencia de deadlocks:**
- **Ausencia de inanición:**
 - Nuestra simplificación, tiene un problema.
 - Eliminó las secciones no críticas, que pueden no terminar.

Algoritmo III (Simplificación)

```
1
2             global turno = 0
3
4  thread p           thread q
5  while(true){       while(true){
6  p0:  while (turno != 0);    q0:  while (turno != 1);
7  p1:  turno = 1             q1:  turno = 0
```

- **Mutex:** Sí
- **Ausencia de deadlocks:** Sí
- **Ausencia de inanición:**
 - Nuestra simplificación, tiene un problema.
 - Eliminó las secciones no críticas, que pueden no terminar.

Algoritmo III (Simplificación)

```
1
2             global turno = 0
3
4  thread p           thread q
5  while(true){       while(true){
6  p0:  while (turno != 0);    q0:  while (turno != 1);
7  p1:  turno = 1             q1:  turno = 0
```

- **Mutex:** Sí
- **Ausencia de deadlocks:** Sí
- **Ausencia de inanición:** Veamos....
 - Nuestra simplificación, tiene un problema.
 - Eliminó las secciones no críticas, que pueden no terminar.

Algoritmo III (Simplificación)

```
1
2         global turno = 0
3
4
5  thread id = 0
6  while(true){
7      // seccion no critica
8      while (turno != id);
9      // seccion critica
10     turno = (id + 1) % 2
11     // seccion no critica
12 }

thread id = 1
while(true){
    // seccion no critica
    while (turno != id);
    // seccion critica
    turno = (id + 1) % 2
    // seccion no critica
}
```

Algoritmo III (Simplificación)

```
1
2             global turno = 0
3
4
5  thread id = 0             thread id = 1
6  while(true){             while(true){
7      // seccion no critica    // seccion no critica
8      while (turno != id);     while (turno != id);
9      // seccion critica       // seccion critica
10     turno = (id + 1) % 2      turno = (id + 1) % 2
11     // seccion no critica     // seccion no critica
12 }
```

- **Mutex:**
- **Ausencia de deadlocks:**
- **Ausencia de inanición:**

Algoritmo III (Simplificación)

```
1
2         global turno = 0
3
4
5  thread id = 0
6  while(true){
7      // seccion no critica
8      while (turno != id);
9      // seccion critica
10     turno = (id + 1) % 2
11     // seccion no critica
12 }

thread id = 1
while(true){
    // seccion no critica
    while (turno != id);
    // seccion critica
    turno = (id + 1) % 2
    // seccion no critica
}
```

- Mutex: Sí
- Ausencia de deadlocks:
- Ausencia de inanición:

Algoritmo III (Simplificación)

```
1
2             global turno = 0
3
4
5  thread id = 0             thread id = 1
6  while(true){             while(true){
7      // seccion no critica    // seccion no critica
8      while (turno != id);     while (turno != id);
9      // seccion critica       // seccion critica
10     turno = (id + 1) % 2      turno = (id + 1) % 2
11     // seccion no critica     // seccion no critica
12 }
```

- Mutex: Sí
- Ausencia de deadlocks: Sí
- Ausencia de inanición:

Algoritmo III (Simplificación)

```
1
2         global turno = 0
3
4
5  thread id = 0                thread id = 1
6  while(true){                while(true){
7      // seccion no critica    // seccion no critica
8      while (turno != id);     while (turno != id);
9      // seccion critica       // seccion critica
10     turno = (id + 1) % 2      turno = (id + 1) % 2
11     // seccion no critica     // seccion no critica
12 }
```

- **Mutex:** Sí
- **Ausencia de deadlocks:** Sí
- **Ausencia de inanición:** No

Algoritmo III (Simplificación)

```
1
2         global turno = 0
3
4
5  thread id = 0           thread id = 1
6  while(true){           while(true){
7      // seccion no critica    // seccion no critica
8      while (turno != id);      while (turno != id);
9      // seccion critica        // seccion critica
10     turno = (id + 1) % 2      turno = (id + 1) % 2
11     // seccion no critica    // seccion no critica
12 }
```

- **Mutex:** Sí
- **Ausencia de deadlocks:** Sí
- **Ausencia de inanición:** No (si el thread id 0 no termina su sección no crítica)

Algoritmo de Dekker (II + III)

```
1
2 shared turno = 0
3 shared flag = {false, false}
4
5
6 thread id = 0                thread id = 1
7 // seccion no critica        ...
8 otro = (id + 1) % 2
9 flag[id] = true
10 while (flag[otro])
11     if (turno == otro)
12         flag[id] = false
13         while (turno != id);
14         flag[id] = true
15
16 // seccion critica
17
18 turno = otro
19 flag[id] = false
20 // seccion no critica
```

Algoritmo de Peterson

```
1
2 shared turno = 0
3 shared flag = {false, false}
4
5
6 thread id = 0
7 // seccion no critica
8 otro = (id + 1) % 2
9 flag[id] = true
10 turno = otro
11 while (flag[otro] && turno == otro);
12
13 // seccion critica
14
15 flag[id] = false
16 // seccion no critica
```

- **Mutex:** Sí
- **Ausencia de deadlocks:** Sí
- **Ausencia de inanición:** Sí

Sólo sirven para dos procesos.

Algoritmo de Bakery

Algoritmo de Bakery

```
1 shared entrando[n] = {false, ..., false}
2 shared numero[n]    = {0, ..., 0}
3
4
5 thread id = 0
6 // seccion no critica
7 entrando[id] = true
8 numero[id] = 1 + max(numero[1], ..., [n])
9 entrando[id] = false
10 for (j = 1; j <= n; ++j)
11     while (entrando[j]);
12     while (numero[j] != 0 && (numero[j] < numero[id] ||
13         (numero[j] == numero[id] && j < id)));
14
15 // seccion critica
16
17 numero[id] = 0
18 // seccion no critica
```

Este algoritmo resuelve el problema para n threads.

Pregunta

¿Y si uno tiene otras acciones atómicas, cómo puede resolver el problema de la exclusión mutua?

Test and set

```
1 function test-and-set(ref comp, ref local)
2     local = comp
3     comp  = 1
```

Test and set

```
1 function test-and-set(ref comp, ref local)
2     local = comp
3     comp = 1

1
2         shared comp = 0
3
4
5 thread id = 0                thread id = 1
6     int local                int local
7     // seccion no critica    // seccion no critica
8     repeat
9         test-and-set(comp, local)
10    until (local == 0)
11    // seccion critica
12    comp = 0
13    // seccion no critica    // seccion no critica
```

Exchange

```
1 function exchange(ref comp, ref local)
2     temp  = comp
3     comp  = local
4     local = temp
```

Exchange

```
1 function exchange(ref comp, ref local)
2     temp  = comp
3     comp  = local
4     local = temp
```

```
1
2         shared comp = 0
3
4
5 thread id = 0                                thread id = 1
6     int local = 1                            int local = 1
7     // seccion no critica                    // seccion no critica
8     repeat                                   repeat
9         exchange(comp, local)                exchange(comp, local)
10    until (local == 0)                        until (local == 0)
11    // seccion critica                        // seccion critica
12    comp = 0                                  comp = 0
13    // seccion no critica                    // seccion no critica
```

Compare-and-swap

```
1 function compare-and-swap(ref comp, ref viejo, ref nuevo)
2     temp = comp
3     if (comp == viejo)
4         comp = nuevo
5     return temp
```

Compare-and-swap

```
1 function compare-and-swap(ref comp, ref viejo, ref nuevo)
2     temp = comp
3     if (comp == viejo)
4         comp = nuevo
5     return temp
```

```
1
2         shared comp = false
3
4
5 thread id = 0                                thread id = 1
6 // seccion no critica                        // seccion no critica
7 while( compare-and-swap(                      while( compare-and-swap(
8     comp, false, true) );                    comp, false, true) );
9 // seccion critica                          // seccion critica
10 comp = false                                comp = false
11 // seccion no critica                        // seccion no critica
```

Fetch-and-add

```
1
2 function fetch-and-add(ref comp, ref local, ref x)
3     local = comp
4     comp  = comp + x
```

Fetch-and-add

```
1
2 function fetch-and-add(ref comp, ref local, ref x)
3     local = comp
4     comp  = comp + x
```

```
1
2             shared ticket = 0
3             shared turno  = 0
4
5
6 thread id = 0             thread id = 1
7     int miTurno           int miTurno
8     // seccion no critica // seccion no critica
9     fetch-and-add(        fetch-and-add(
10         ticket, miTurno, 1) ticket, miTurno, 1)
11     while (turno != miTurno); while (turno != miTurno);
12     // seccion critica    // seccion critica
13     fetch-and-add(        fetch-and-add(
14         turno, miTurno, 1) turno, miTurno, 1)
15     // seccion no critica // seccion no critica
```

Busy waiting

Todas las soluciones vistas en esta clase son ineficientes dado que consumen tiempo de procesador en las esperas.

Sería deseable suspender la ejecución de un proceso que intenta acceder a la sección crítica hasta tanto sea posible.